University of Primorska
Faculty of Mathematics, Natural Sciences and Information Technologies

PROJECT REPORT

Arithmetic Coding vs Huffman Coding

A Comparative Study of Lossless Data Compression Algorithms

Authors

Viktor Taseski | 89231157

Mario Krajoski | 89231110

Course: Information Theory and Coding

Academic Year: 2025/2026

Date: May 2026

Table of Contents

1. Introduction

Data compression is a fundamental area of information theory concerned with representing information using fewer bits than the original encoding. Lossless compression algorithms are particularly important in scenarios where the original data must be perfectly reconstructed from the compressed representation, such as in file archiving, text transmission, and source coding applications.

This project implements and compares two classical lossless compression algorithms: Arithmetic Coding and Huffman Coding. Both algorithms are entropy coders — they exploit the statistical properties of a source to achieve compact representations. While Huffman coding assigns a distinct binary codeword to each symbol, arithmetic coding encodes an entire message as a single high-precision fractional number, theoretically achieving a compression ratio closer to the theoretical entropy limit.

The source alphabet used throughout this project consists of four symbols {a, b, c, d} with fixed probability distributions: P(a) = 0.4, P(b) = 0.3, P(c) = 0.2, P(d) = 0.1. The program generates a random sequence of 100 symbols drawn from this distribution and measures the compressed output size in bits for both methods, comparing the results against the theoretical entropy bound.

2. Theoretical Background

2.1 Information Entropy

The foundation of both compression methods is Shannon's entropy, which defines the theoretical lower bound on the average number of bits required to encode a symbol from a given probability distribution. For a source with n symbols, the entropy H is given by:

$$H = -\Sigma\ P(x_i)\ \cdot \log_2(P(x_i))$$

For the probability distribution used in this project (P(a)=0.4, P(b)=0.3, P(c)=0.2, P(d)=0.1), the entropy evaluates to:

$$H = -(0.4 \cdot \log_2(0.4) + 0.3 \cdot \log_2(0.3) + 0.2 \cdot \log_2(0.2) + 0.1 \cdot \log_2(0.1)) \approx 1.846 \text{ bits/symbol}$$

For a sequence of 100 symbols, the entropy lower bound is therefore approximately 184.6 bits. Any lossless encoder must use at least this many bits on average to represent the sequence perfectly.

2.2 Arithmetic Coding

Arithmetic coding is a form of entropy coding that represents an entire message as a single real number in the interval [0, 1). Rather than replacing each symbol with a specific codeword, arithmetic coding encodes the entire message into a single fractional number, achieving a compression rate much closer to the theoretical entropy.

The algorithm operates by maintaining a current interval [low, high), initially set to [0, 1). For each symbol in the message, the interval is subdivided according to the

cumulative probability distribution of the source alphabet. The sub-interval corresponding to the observed symbol becomes the new current interval.

The cumulative intervals for the source alphabet used in this project are defined as follows:

Symbol	Probability	Interval Start	Interval End
a	0.4	0.0	0.4
b	0.3	0.4	0.7
c	0.2	0.7	0.9
d	0.1	0.9	1.0

After encoding, the final interval [low, high) narrows with each symbol processed. The encoded value is the midpoint of this final interval: `code = (low + high) / 2`. The number of bits required to represent this value is estimated as `ceil(-log2(high - low))`, corresponding to the precision needed to uniquely identify the interval.

2.3 Arithmetic Decoding

Decoding reverses the encoding process. Given the encoded value and the original message length, the decoder identifies which symbol's interval contains the current value, appends that symbol to the decoded message, and rescales the value to narrow the interval, mirroring the encoding steps exactly.

Due to the recursive nature of the algorithm and the extremely narrow intervals produced by long messages, high-precision arithmetic is required. This implementation uses Python's Decimal module with a precision of 300 significant digits to avoid floating-point underflow and ensure correct decoding for 100-symbol messages.

2.4 Huffman Coding

Huffman coding is a variable-length prefix-free coding algorithm that assigns shorter codewords to more probable symbols and longer codewords to less probable ones. It was developed by David A. Huffman in 1952 and remains one of the most widely used compression schemes in practice.

The Huffman tree is constructed greedily using a min-heap (priority queue). Initially, each symbol is a leaf node with weight equal to its probability. The two nodes with the lowest frequencies are repeatedly merged into a parent node until only one root node remains. The binary codeword for each symbol is then read off as the path from the root to the leaf.

For the given probability distribution, the Huffman codes are typically assigned as follows (exact assignment may vary by implementation due to tie-breaking):

Symbol	Probability	Huffman Code	Code Length (bits)
a	0.4	0	1
b	0.3	10	2
c	0.2	110	3
d	0.1	111	3

The average Huffman codeword length for this distribution is: $L = 0.4\times1 + 0.3\times2 + 0.2\times3 + 0.1\times3 = 0.4 + 0.6 + 0.6 + 0.3 = 1.9$ bits/symbol. For 100 symbols, this gives approximately 190 bits, which is slightly above the entropy of 184.6 bits.

2.5 Comparison of Both Methods

The key theoretical difference between the two methods is granularity. Huffman coding assigns a whole number of bits to each symbol, meaning it cannot achieve rates below 1 bit/symbol even for very probable symbols. Arithmetic coding, by contrast, effectively assigns a fractional number of bits to each symbol by encoding the entire message jointly, making it capable of approaching the entropy limit arbitrarily closely.

In practice, arithmetic coding requires careful implementation to handle the high-precision arithmetic required for long sequences, while Huffman coding is simpler, faster, and well-suited for hardware implementations. Both are used in real-world applications: Huffman coding in ZIP, JPEG, and MP3; arithmetic coding in JPEG 2000 and H.264 video compression.

3. Implementation

3.1 Project Structure

The entire project is implemented as a single Python script: index.py. The script is self-contained, using only Python's standard library (random, math, heapq, decimal) for the arithmetic coding portions and a custom Huffman coding implementation built from scratch. No external libraries are required.

The program is organized into the following logical sections:

- Probability definition and cumulative interval construction
- Random sequence generation (100 symbols)
- Arithmetic encoding function
- Arithmetic decoding function
- Huffman tree construction using a min-heap
- Huffman code generation via tree traversal
- Output and comparison of both methods

3.2 High-Precision Arithmetic

A critical design decision in the implementation is the use of Python's Decimal module for arithmetic coding. When encoding a 100-symbol sequence, the interval [low, high) shrinks to an extremely small width — on the order of 10^-88 for this source. Standard 64-bit floating-point numbers cannot represent such values without catastrophic precision loss, leading to decoding failures.

The implementation sets the Decimal precision to 300 significant digits, which is more than sufficient for sequences of up to several hundred symbols. This ensures that the encoded value and the decoded sequence are always identical to the original.

```
from decimal import Decimal, getcontext
getcontext().prec = 300
```

3.3 Arithmetic Encoding Algorithm

The `arithmetic_encode()` function processes each symbol in the input message one by one. It maintains two Decimal variables, low and high, initialized to 0 and 1 respectively. For each symbol, the current range (high - low) is scaled according to the symbol's cumulative probability interval:

```
high = low + range_width * sym_high
low  = low + range_width * sym_low
```

At the end, the function returns the midpoint of the final interval as the encoded value, along with the interval bounds. The number of bits required to represent the encoded value is estimated as ceil(-log2(high - low)), which reflects the precision needed to distinguish the final interval from all others.

3.4 Arithmetic Decoding Algorithm

The `arithmetic_decode()` function reconstructs the original sequence from the encoded value and the sequence length. It iterates length times, maintaining the same low and high variables as the encoder. At each step, it normalizes the encoded value within the current interval and identifies which symbol's probability sub-interval contains it:

```
value = (code - low) / range_width
if sym_low <= value < sym_high:  # found the symbol
    decoded += symbol
```

After identifying the symbol, the decoder rescales the interval exactly as the encoder did, maintaining perfect synchrony with the encoding process. Correctness is verified by comparing `decoded_sequence == sequence`, which must evaluate to True.

3.5 Huffman Coding Implementation

The Huffman coder is implemented from scratch using Python's `heapq` module for the priority queue. Two classes drive the implementation: a Node class representing tree nodes, and two functions: `build_huffman_tree()` to construct the tree and `generate_codes()` to extract the binary codewords.

`build_huffman_tree()` initializes a min-heap with one leaf node per symbol, weighted by probability. It then repeatedly pops the two lightest nodes, merges them into a new parent node with combined weight, and pushes the parent back into the heap. This continues until a single root node remains.

`generate_codes()` performs a depth-first traversal of the tree. Left branches append '0' and right branches append '1' to the current path string. Leaf nodes store the final codeword in a dictionary. The encoded message is produced by concatenating the codewords for all symbols in the input sequence.

4. Results and Analysis

4.1 Sample Output

Running the program on a 100-symbol sequence produces output structured into three parts: the generated sequence, the arithmetic coding output (encoded value and decoded verification), and the Huffman output (codes and bit count).

A representative sample output from the program is shown below:

```
Generated sequence:
aabacbaadbacaabcaababcabaacbaaabacaabaabc...(100 symbols)
Decoded correctly: True
Estimated arithmetic coding bits: 186
Huffman encoded length: 191 bits
Arithmetic coding is better.
```

4.2 Compression Comparison

Over multiple runs with different randomly generated sequences, arithmetic coding consistently produces fewer bits than Huffman coding. Both methods stay within a few percent of the theoretical entropy bound of approximately 184.6 bits for 100 symbols.

Method	Typical Bits (100 symbols)	Gap Above Entropy
Entropy Bound	~185	0 bits
Arithmetic Coding	185–190	0–5 bits
Huffman Coding	188–194	3–9 bits

The gap between arithmetic coding and the entropy bound is primarily due to the rounding in the `ceil()` operation when estimating bit count. In a true binary transmission, the final encoded value must be represented with enough binary digits to uniquely identify the interval, typically adding 1-2 extra bits beyond the theoretical minimum.

4.3 Correctness Verification

The correctness of the arithmetic coder is verified by the condition `decoded_sequence == sequence`. This comparison checks that every one of the

100 symbols in the decoded output exactly matches the original. Thanks to the 300-digit Decimal precision, this check reliably returns True across all test runs.

Huffman coding is inherently correct by construction — the prefix-free property of the code guarantees that any encoded bitstring can be unambiguously decoded back to the original symbols by traversing the Huffman tree.

5. Discussion

5.1 Strengths and Limitations

Arithmetic coding achieves a compression ratio very close to the theoretical entropy bound, making it the more efficient method from an information-theoretic perspective. However, it requires high-precision arithmetic, which incurs significant computational overhead compared to Huffman coding. In this implementation, the use of Python's Decimal module with 300-digit precision resolves correctness issues but is slower than native floating-point operations.

Huffman coding, on the other hand, is extremely simple, fast, and hardware friendly. Its main limitation is the one-integer-bit-per-symbol constraint, which means it cannot use fractional bits and always leaves a small gap above entropy. For this particular source, the average codeword length of 1.9 bits/symbol is very close to the entropy of 1.846 bits/symbol, so the performance gap is small.

5.2 Practical Considerations

In real-world applications, several additional factors affect the choice of compression algorithm. Huffman coding requires transmitting the codebook (tree structure) along with the compressed data, which adds overhead for short messages. Arithmetic coding does not require a codebook, since the probability model is known a priori to both encoder and decoder.

For adaptive or streaming scenarios, arithmetic coding is also more flexible: the probability model can be updated symbol-by-symbol as new data is observed, whereas Huffman trees require rebuilding when the distribution changes. This adaptability makes arithmetic coding the backbone of modern compressors such as those used in CABAC (in H.264/H.265 video).

5.3 Symbol Independence Assumption

Both algorithms in this project assume that the symbols are generated independently (i.i.d.). In practice, real data sources exhibit strong statistical dependencies between symbols — text, for instance, has very predictable character sequences (e.g., 'q' is almost always followed by 'u' in English). Exploiting these higher-order dependencies with context modelling can dramatically improve compression ratios beyond what simple symbol-by-symbol entropy coding achieves.

6. Conclusion

This project successfully implemented and compared two fundamental lossless data compression algorithms: Arithmetic Coding and Huffman Coding. Both were applied to a 100-symbol sequence drawn from a four-symbol source with known probability distribution.

The results confirm the theoretical predictions: arithmetic coding produces fewer bits than Huffman coding and comes closer to the Shannon entropy bound. Correctness of the arithmetic coder was verified by exact decoding of all test sequences, enabled by the use of Python's high-precision Decimal arithmetic with 300 significant digits.

Huffman coding, while slightly less efficient in terms of compression ratio, is simpler to implement and significantly faster. For the given source, both methods perform well, with the gap to entropy being only a few bits per 100 symbols.

The project illustrates a key insight from information theory: there is no universally best compression algorithm. The right choice depends on the trade-off between compression ratio, computational complexity, implementation simplicity, and the specific properties of the data source.

7. References

[1] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal,* 27(3), 379–423.

[2] Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE,* 40(9), 1098–1101.

[3] Witten, I. H., Neal, R. M., & Cleary, J. G. (1987). Arithmetic Coding for Data Compression. *Communications of the ACM,* 30(6), 520–540.

[4] Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory* (2nd ed.). Wiley-Interscience.